中国大学生计算机设计大赛

软件开发类作品文档简要要求

作品编号：2026054264

作品名称：云枢观策

版本编号：V1.0

填写日期：2026 年 4 月 20 日

填写说明：

1、本文档适用于所有涉及软件应用与开发类的各个小类作品，包括：（1）Web 应用（2）移动应用（非游戏类）（3）算法设计与应用类程序（4）工具软件（5）软件应用与开发专项赛；

2、本文档为简要文档，不宜长篇大论，需简明扼要，建议设计二级目录，逻辑性强；

3、一级标题采用二号黑体，居中，二级标题采用三号黑体，靠左，根据需要可以设计三级标题，正文一律用五号宋体；

4、提交文档时，以 PDF 格式提交本文档；

5、本文档内容是正式参赛内容组成部分，务必真实填写。如不属实，将导致奖项等级降低甚至终止本作品参加比赛。

6、项目中涉及应用人工智能大模型的内容，务必在各个部分从以下几方面说明。

（1）AI 使用说明

- 使用了哪些模型/工具（名称/版本/日期）
- 在哪些环节应用了 AI：构思、生成、润色、编程、数据分析、图像/音频生成等
- 学生做了哪些把关工作？AI 生成的结果的正确性如何验证？

（2）关键提示词与输出样例

- 例举最重要的、决定性的 5-10 条对话

（3）版权说明

- 提交 AI 生成涉及的引用列表、素材授权说明（若有）

目　录

第一章　需求分析……3
第二章　概要设计……5
第三章　详细设计……7
第四章　测试报告……15
第五章　安装及使用……15
第六章　项目总结……21
参考文献……22

第一章 需求分析

云枢观策(CloudPolicy)面向的是复杂公共信息的理解与后续承接问题。现实中，社区通知、校园事务说明、惠企政策、办事指南等内容常以网页公告、PDF 附件、截图、扫描件和表格等多种形式出现，信息来源分散、文本冗长、材料要求隐蔽、执行步骤不清晰，用户往往需要反复阅读、手工整理，才能判断“是否适用、需要准备什么、下一步该做什么”。现有方式要么停留在公告浏览与搜索层面，要么仅提供通用问答，缺少对多模态材料、可追问解释、结果沉淀和后续执行链路的统一支持。

本作品的目标是构建一套面向真实使用过程的智能化服务平台，将“材料接入、内容理解、继续追问、结果沉淀、反馈回流”连接起来。系统支持 PDF、DOCX、XLSX、图片、

截图、URL 等多种材料接入，通过自由格式图谱解析、RAG 检索增强和可解释智能体，将复杂内容转化为更适合阅读和执行的结果。普通用户可利用系统完成理解、收藏、待办和追问；认证主体可发布文档并追踪反馈；管理员可审核内容并查看全站统计。作品的核心需求不是单纯“把内容总结短一点”，而是提供一条可持续使用的理解与承接链路。

方式	典型使用方式	主要优势	主要局限	是否支持多模态材料	是否支持追问	是否支持结果沉淀
传统公告阅读	用户自行阅读网页公告、PDF 通知、截图或纸质材料	原始信息完整，来源直接	文本冗长、理解成本高、重点不突出，难以快速定位适用对象、所需材料和办理步骤	弱	否	否
通用关键词搜索	借助站内搜索、浏览器搜索或搜索引擎查找相关信息	查找入口简单，适合已知关键词场景	结果分散，语义理解弱，常需要用户自行二次筛选和整合	弱	否	否
普通聊天式 AI 问答	将材料复制给通用大模型进行概括或问答	交互自然，能够快速生成摘要和解释	对附件、图文混排材料和来源追溯支持有限，回答易停留在一次性问答层面	中	是	弱
云枢观策闭环方案	上传文本、PDF、DOCX、	支持多模态接入、自由格式图	系统模块较完整，对部署环境和数	是	是	是

	XLSX、图片、截图、URL 等材料，进入解析、追问、收藏、待办、反馈与管理闭环	谱解析、RAG 检索增强、智能体追问、结果沉淀和反馈回流，形成完整使用链路	据组织有一定要求			

表 1-1 替代方式对比表

第二章 概要设计

2.1 总体设计思路

作品围绕“看得懂、找得到、办得成、可反馈、可沉淀”的目标展开设计。系统采用展示入口与业务入口并行的双入口结构：

1. 展示首页用于作品展示、答辩说明和整体能力呈现；
2. 业务系统用于真实上传、解析、追问、收藏、待办、发布和管理。

通过这种方式，作品既具备完整展示能力，又保留可运行的产品结构。

2.2 功能模块设计

系统主要由六个模块组成：

1. 多模态解析工作台：负责上传 PDF、Word、Excel、图片、截图和 URL 等材料，并显示解析进度与结果。
2. 智能体系统：负责持续追问、RAG 检索增强、推理轨迹展示和会话管理。

3. 图谱结果与原文联动：负责将解析结果组织为自由格式图谱，并支持原文回看。

4. 结果沉淀模块：包括收藏、历史记录和待办事项，承接后续使用。

5. 发布与反馈模块：包括政策发布、公开展示、评价、纠错和留言。

6. 管理与分析模块：包括用户管理、审核管理、全站统计和展示大屏。

2.3 创新设计概述

本作品的首要创新点是**闭环设计**。系统不是只提供一次性解析，而是把“上传材料、完成解析、继续追问、形成结果、收藏与待办、公开反馈、后台观察”连成一条完整链路，使一次解析能够持续服务后续使用。

第二层创新点在于支撑闭环的三项核心能力：

1. 自由格式图谱解析：不依赖固定字段模板，而是通过 dynamic_payload 反推 nodes / links，更适应复杂材料。

2. 可解释智能体：不仅给出回答，还展示实时推理轨迹和证据片段。

3. 多模态统一接入：不同材料走不同提取路径，但最终进入同一理解链路和结果体系。

第三层创新点体现在具体技术实现上，包括 RAG 证据增强、轨迹先行的流式编排、统计建模与双入口产品结构。这些能力共同服务于前述闭环，而不是孤立堆叠。

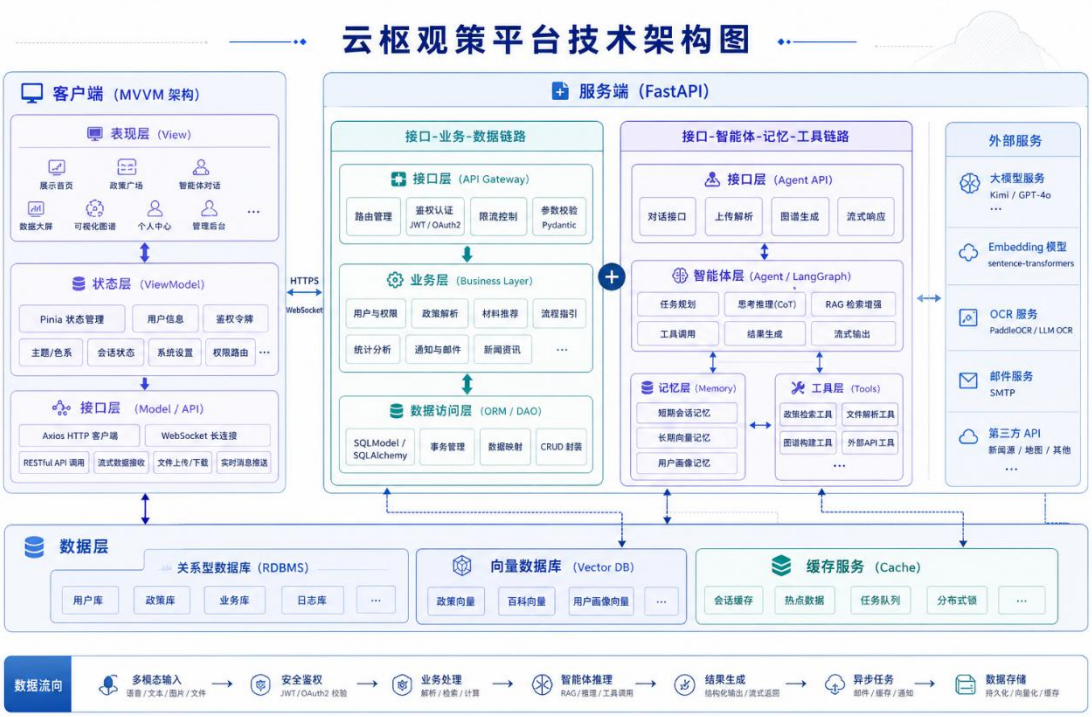

图 2-1 系统总体架构图

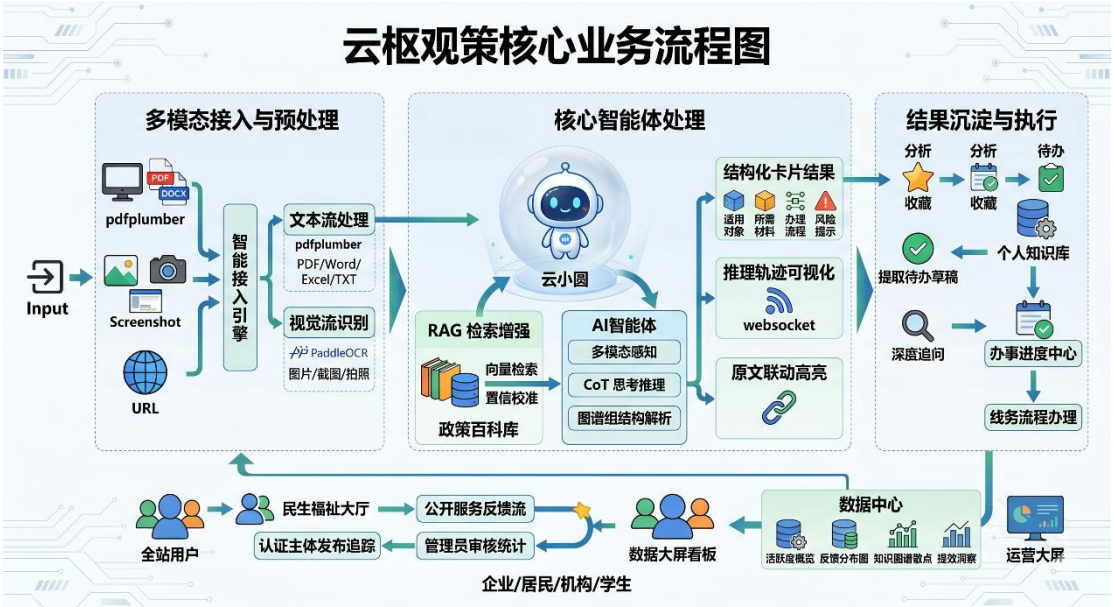

图 2-2 核心业务流程图

第三章 详细设计

3.1 界面设计

系统界面分为展示层和业务层两部分。展示层包括展示首页、展示版发现页和展示大屏，

主要用于概览作品定位、核心亮点和全站数据。业务层包括多模态解析主页、智能体页、政策广场、收藏页、历史页、待办页、发布中心、反馈大厅和管理员后台，强调真实使用过程中的连续操作。

多模态解析主页负责材料上传、结果展示和示例材料承接；智能体页负责多轮追问、证据查看和轨迹展示；图谱页面用于查看节点关系、原文映射和重点信息；管理员后台用于查看用户结构、审核结果和统计分析。界面设计的重点不在于单页视觉效果，而在于把不同业务动作组织成连续流程。

图 3-1 多模态解析图谱

图 3-2 智能体与推理轨迹界面图

图 3-3 公共数据分析大屏

3.2 系统架构设计

前端采用 Vue 3 + Vite + Pinia + Vue Router 组合，负责页面渲染、全局状态管理和路由守卫。

后端采用 FastAPI + SQLModel + SQLite + Redis 组合，负责接口服务、会话管理、上传处理和统计分析；智能中枢由文档解析、OCR、RAG 和 Agent 插件组成；部署层采用 Docker Compose + Nginx，负责前后端服务编排与代理转发。

系统通过 REST API 和 WebSocket 两类接口承接核心业务。普通解析任务由上传接口触发，智能体连续对话由 /agent/ws 承接。媒体资源统一挂载在 /media 路径下，Nginx 负责代理静态资源、API 路由和 WebSocket 连接。

3.3 数据库设计

数据库设计围绕用户、内容、会话、反馈和统计五类对象展开。

表名	主要作用	关键字段	关系说明
User	用户身份与角色管	uid（主键）、uname（用户名）、email（邮箱）、phone（手机号）、role	与设置、解析记录、会话、发布文

	理	（角色）、profession（职业标签）、created_time（创建时间）	档、反馈、收藏、待办等核心业务表关联
Settings	用户个性化设置	id（主键）、user_id（用户编号）、default_audience（默认受众）、theme_mode（主题模式）、color_scheme（色系方案）、system_notifications（通知开关）	与 User 一对一，用于界面偏好和默认行为配置
ChatMessage	解析结果主表	id（主键）、user_id（用户编号）、original_text（原始文本）、file_url（源文件地址）、required_materials（所需材料）、risk_warnings（风险提醒）、chat_analysis（分析数据）、created_time（创建时间）	与用户、收藏、待办、历史记录关联，是解析链路的核心数据表
AgentConversation	智能体会话管理	id（主键）、user_id（用户编号）、title(会话标题）、created_time(创建时间）、updated_time（更新时间）	与 User、AgentMessage 关联，用于恢复和管理多轮对话
AgentMessage	智能体消息记录	id（主键）、conversation_id（会话编号）、user_id（用户编号）、role（消息角色）、content(消息内容）、	属于某个会话，用于保存完整消息轨迹

		created_time（创建时间）	
PolicyDocument	政策文档发布与审核	id（主键）、uploader_id（发布者编号）、title（标题）、content（正文）、category（分类）、status（审核状态）、view_count（浏览量）、created_time（创建时间）	与发布者和反馈表关联，支撑发布、审核、公开展示与推荐
Opinion	用户反馈信息	id（主键）、doc_id（文档编号）、user_id（用户编号）、opinion_type（反馈类型）、content（反馈内容）、rating（评分）、like_count（点赞量）、created_time（创建时间）	与 PolicyDocument、User 关联，支撑评价、纠错和留言链路
TodoItem	办事进度与待办事项	id（主键）、user_id（用户编号）、source_chat_id（来源解析记录）、title（待办标题）、deadline（截止时间）、is_done（完成状态）、is_confirmed（确认状态）、created_time（创建时间）	与用户及解析结果关联，用于承接“看懂之后继续办”
Favorite	收藏沉淀	id（主键）、user_id（用户编号）、chat_message_id（解析结果编号）、note（备注）、created_time（创建时间）	与用户及解析结果关联，用于长期沉淀重点内容
HistoryEvent	历史轨迹	id（主键）、user_id（用户编号）、	支撑历史恢复、行

	记录	domain（业务域）、event_type（事件类型）、title（标题）、route_path（跳转路径）、occurred_time（发生时间）	为追踪和跨页面回溯
StatsAnalysis	统计分析结果	id（主键）、user_id（用户编号）、materials_freq（材料词频）、risks_freq（风险词频）、complexity_distribution（复杂度分布）、total_time_saved_minutes（累计节省时间）、updated_time（更新时间）	支撑个人中心、管理后台和展示大屏的统计分析

表 3-1 关键数据表说明表

系统的数据关系主链路为：

1. User -> ChatMessage -> Favorite / TodoItem / HistoryEvent，

2. User -> AgentConversation -> AgentMessage，

3. User -> PolicyDocument -> Opinion。

这种设计既保证主业务链路清晰，也为搜索、推荐和统计功能预留了扩展空间。

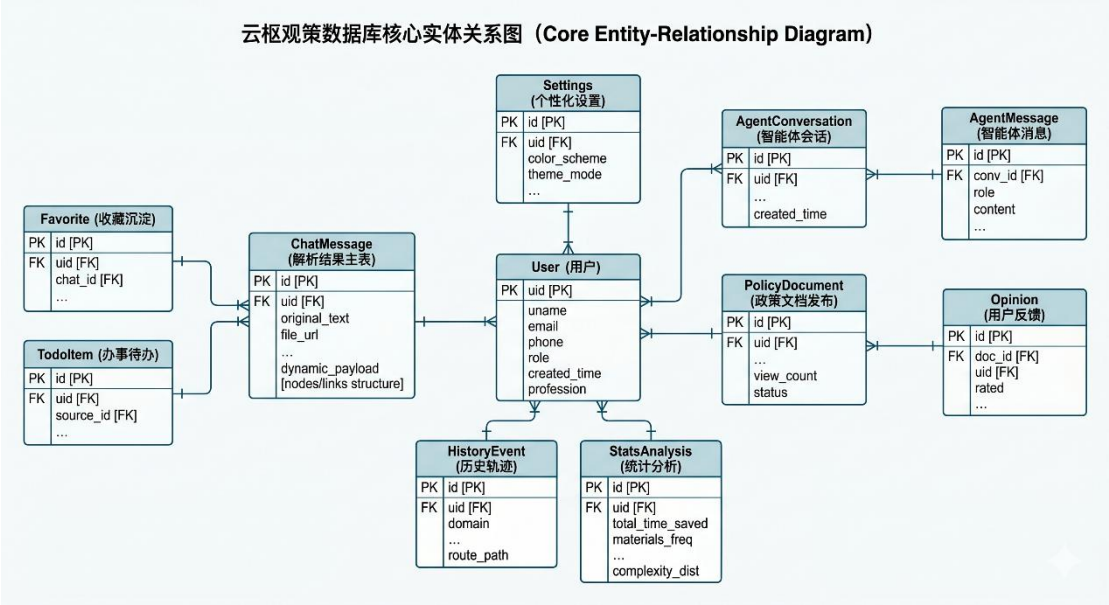

图 3-4 数据库核心实体关系图

3.4 关键算法与技术设计

3.4.1 自由格式图谱解析

系统采用“严格约束输出 + 宽松修复 + Repair-Only 二次修复 + 可展示回退”的解析策略。模型优先输出 JSON 对象，再由后端生成 `dynamic_payload` 并反推为 `nodes / links`。当模型输出不规范时，系统执行对象提取、冗余逗号修复和截断补全，尽可能保留有效信息。其价值在于让复杂材料以图谱形式表达，而不是被强行压缩为固定字段模板。

3.4.2 语义检索与置信校准

系统使用本地 embedding 模型与 ChromaDB 维护知识库，在回答前执行语义检索，并通过 top_k + score_threshold 双阈策略过滤低质量片段。为估计结果置信度，系统采用如下模型：

$$C = \min(0.95, 0.35 + 0.2 \cdot I(\text{parse_mode} = \text{ai}) + 0.45 \cdot \min(\bar{s}, 1))$$

其中 C 表示最终置信度，$\bar{s}$ 表示证据平均相似度分数。该模型同时考虑解析方式和证据命中情况。

3.4.3 图谱侧推荐评分

系统在图谱工作区中根据职业、角色、可执行信号、时效约束和复杂度节省价值计算推荐分数：

$$R = 100 \cdot \frac{w_p S_{profession} + w_r S_{role} + w_a S_{action} + w_u S_{urgency} + w_c S_{complexity}}{\sum w}$$

该公式使推荐不依赖黑盒排序，而能根据用户画像和文档内容显式给出分数依据。

3.4.4 实时轨迹调度

WebSocket 交互中，后端通过 asyncio.Queue + asyncio.to_thread 形成双通道调度模型，将思考步骤、工具调用、正文结果分阶段推送；前端通过最小可读时长策略控制轨迹消费节奏，避免轨迹闪过或被正文覆盖。

3.4.5 复杂度与效率建模

系统利用图谱节点规模、关系密度和原文长度构造复杂度与效率指标。综合难度因子定义为：

$$D = \alpha \cdot \ln\left(W_{original}\right) + \beta \cdot N_{entities} + \gamma \cdot \sigma_{logic}$$

时间节省估算模型为：

$$T_{saved} = \max\left(0, \frac{W_{original}}{V_{human}} - T_{card}\right)$$

在全站层面，还定义协同效率指数：

$$\mathcal{E} = \sum_{i=1}^{n} T_{saved,i} \cdot \omega_{user}$$

这些指标用于支撑个人中心、管理后台和展示大屏中的趋势分析。

3.5 重点难点

1. 自由格式图谱解析难点：材料结构差异大，固定模板容易丢失关系信息。系统通过 JSON 约束输出与多级修复机制保证结果可展示。

2. 实时轨迹展示难点：推理速度与前端渲染节奏不一致。系统通过队列解耦和分阶段推送保持轨迹可读。

3. 多模态文档处理难点：图文混排 PDF、截图和扫描件处理路径不同。系统采用文本优先、

OCR 兜底和结果标准化策略。

4. 可信度保障难点：通用问答结果难以核对。系统通过知识库检索、证据展示和 RAG 指标统计进行控制。

5. 结果沉淀难点：若无收藏、待办和历史链路，解析结果难以长期复用。系统通过多表协同实现持续承接。

6. 输出安全与敏感数据过滤: Agent 输出的内容参差不齐,对于用户传递的敏感数据需要更多的审核和审查。系统通过”敏感词库+LLM 意图拦截”的节点注入,保证了 Agent 的输出内容安全性。

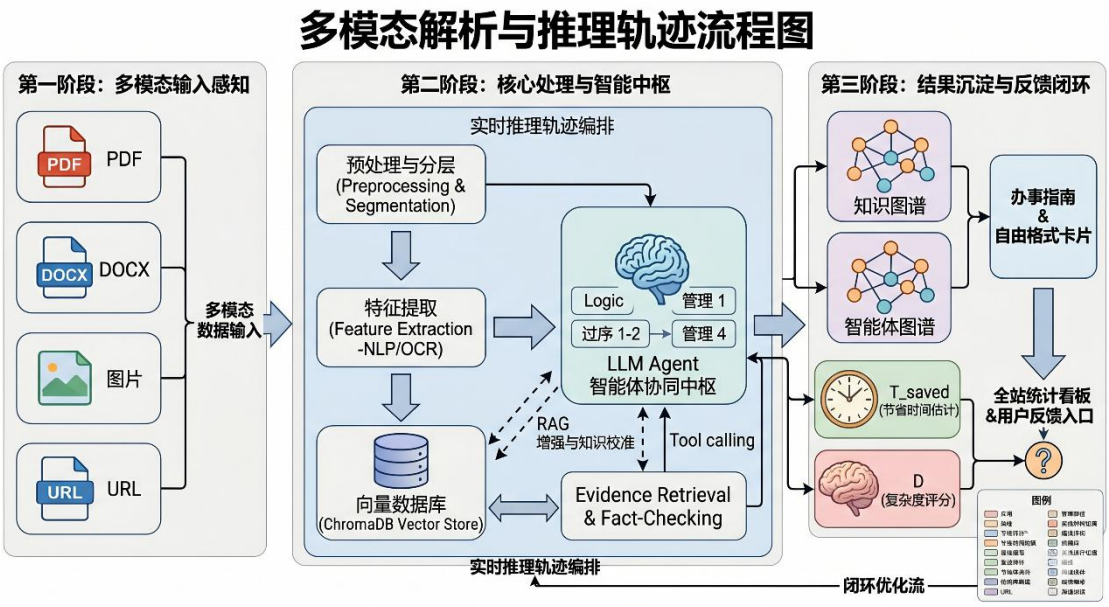

图 3-5 多模态解析与推理轨迹流程图

第四章 测试报告

4.1 测试目标与方法

测试工作围绕系统核心业务链路、关键技术链路和部署运行情况展开，重点验证多模态接入、自由格式图谱结果、智能体交互、结果沉淀、内容发布与审核、反馈链路和管理后台等能力。

验证方式以功能验证、场景验证和部署验证为主。

4.2 核心功能验证

编号	验证项目	验证内容	结果
1	文档上传解析	上传 PDF、DOCX、TXT 并进入解析链路	通过
2	图片与截图 OCR	上传图片、截图材料并提取文字	通过
3	自由格式图谱结果展示	结果以图谱、重点信息和原文联动方式呈现	通过
4	智能体对话	在 /agent 页面进行多轮追问	通过
5	实时推理轨迹	轨迹通过 WebSocket 分阶段显示	通过
6	历史记录恢复	历史记录可查看、恢复和继续使用	通过
7	收藏与待办	解析结果可收藏并生成待办事项	通过
8	政策发布审核	认证主体发布、管理员审核、公开展示	通过
9	反馈链路	用户可提交评价、纠错和留言	通过
10	管理后台	用户管理、统计分析和日志查看	通过

4.3 典型场景与容错验证

编号	场景	验证重点	结果
1	OCR 文本标准化	图片、截图提取后的文本能够进入同一解析链路	通过
2	输出异常修复	模型输出异常时仍可保留最小可用结果	通过
3	长文档处理	解析过程具备进度和状态反馈	通过
4	轨迹与正文分离	轨迹不会被正文输出覆盖	通过

5	会话历史恢复	再次进入页面后仍可读取既有会话	通过
6	Docker 访问链路	前端、后端、WebSocket 与媒体文件均可访问	通过

4.4 技术指标归纳

维度	**当前表现**
输入覆盖范围	支持 PDF、DOCX、DOC、XLSX、XLS、TXT、图片、截图和 URL
结果形态	支持自由格式图谱结果、重点信息回显和原文联动
交互解释性	支持实时推理轨迹、证据展示和历史恢复
结果沉淀能力	支持收藏、待办、历史记录和公开反馈
多角色协同	支持普通用户、认证主体和管理员三类角色
部署方式	支持本地运行与 Docker Compose 部署

综合验证结果表明，系统已完成核心链路的功能验证与部署验证，具备稳定展示和继续扩展的基础。

第五章 安装及使用

.env 环境变量配置（请在项目根目录创建）：详见 README.md 中关于.env 配置.

关于云服务器部署的章节,这里不多赘述.

5.1 运行

5.1.1 前端(web)

cd web

npm install --verbose

npm run dev

5.1.2 后端(app)

1. 启动之前:部署好 Redis,可采用 docker 进行部署.
    1. 未安装 Redis 时：开启 Docker Desktop 后,在终端输入：**docker run --name redis -p 6379:6379 -d redis**
    2. 安装后:
    - 停止 Redis： **docker stop redis**
    - 启动 Redis： **docker start redis**
2. 配置环境变量:在根目录创建.env,写入 README.md 中所要求的环境变量字段.
3. 创建虚拟环境:

   先保证系统已安装 Python 3.12 ，并已将 Python 添加到系统 PATH 中。

   然后在终端输入:

   Windows: **python -m venv .venv**

   macOS/Linux: **python3 -m venv .venv**
4. 激活虚拟环境:

   Windows: **.venv\Scripts\activate**

   macOS/Linux: **source .venv/bin/activate**
5. 安装依赖:

pip install -r app/requirements.txt

5. 启动后台服务:

uvicorn app.main:app --reload

5.2 Docker 部署

使用 Docker 可以极大地简化环境配置和项目启动过程。请确保您的系统已安装 Docker

和 Docker Compose。

并在 Docker Desktop - Settings - Docker Engine 中配置镜像源(详见 README.md)

1. 在 Docker 部署前，必须先准备好本地 Embedding(强制)；如果你要启用文档图片 OCR，也可以选择性准备好本地 PaddleOCR 模型(不强制,不影响运行)。容器启动时只会检查并直接报错，不会在容器内自动下载模型。可以先在本地跑一次后端，或单独执行以下脚本：

   - **python -m app.scripts.download_embedding (必须执行)**
   - **python -m app.scripts.download_paddleocr_models (可以不执行)**

   说明：

   - 执行 paddleocr 脚本需要 3.12 版本的 python.
   - 如果部署环境无法顺利下载或预热 paddleOCR 模型，也可以先关闭 PADDLEOCR_ENABLED，保留 LLM OCR 作为备选链路。
   - 执行上述脚本后，app/resources/paddleocr 与 app/resources/paddlex/official_models 都会在项目目录内完成准备。

2. **启动服务**:

   在项目根目录执行以下命令：

   docker-compose up --build -d

3. **访问应用**:

   - 可以在 docker-compose 完成构建后在浏览器中访问 **http://localhost** 访问应用.
   - 前端应用将通过 Nginx 运行在 http://localhost:80。

- 后端 API 将在 Docker 内部运行于 http://app:8080 ,并通过前端 Nginx 代理访问。
- API 文档可通过 http://localhost/api/docs、http://localhost/api/redoc、http://localhost/api/openapi.json 与 http://localhost/api/openapi.yaml 访问。
- 也可以打开 Docker Desktop 找到 CloudPolicy 容器,即可运行,点击 web 的链接即可打开网页.

4. **停止服务**:在项目根目录执行：

   docker-compose down

   再次启动服务:在项目根目录执行:

   docker-compose up

5.3 构建

5.3.1 后端生产环境构建

后端不使用 pyinstaller 打包：本项目依赖 PaddleOCR、PyTorch、ChromaDB、LangChain 等重型库，打包体积超过 1GB；向量数据库、embedding 模型、uploads/ 等运行时资源无法静态内嵌；建议使用 Docker 部署。

5.3.2 前端生产环境构建

cd web

npm install --verbose

npm run build

构建完成后，将在 web/dist 目录下生成静态文件，可以直接将其部署至 Nginx 的 html/ 目录.

若集成到后端: 将 dist 内的所有内容拷贝至后端项目的静态文件目录，并在 FastAPI 中挂

载：

app.mount("/", StaticFiles(directory="dist", html=True), name="static")

第六章 项目总结

6.1 克服的困难

项目推进过程中，团队首先面对的是“输入不稳定、输出要可用”的矛盾。实际材料既包含通知公告、政策解读、流程说明，也可能包含截图、图片和非标准文本，格式差异大、表述方式不统一，难以直接套入固定模板。为解决这一问题，系统没有采用单一字段抽取思路，而是构建了自由格式图谱解析链路，通过模型约束输出、后处理修复、证据回指和结构重建，尽可能在复杂输入条件下保留有效信息。

第二个难点是“能回答”与“能解释”之间的平衡。仅给出结论难以支撑后续使用，因此系统在问答之外增加了证据检索、来源标注、图谱展示、历史轨迹和结果沉淀机制，使用户不仅能看到结论，还能追溯结论从何而来、可继续执行什么事项。这个过程涉及前端交互、后端接口、检索模块和智能体流程的协同，也是本项目的核心实现难点之一。

第三个难点来自工程落地。项目采用前后端分离架构，涉及数据库设计、文件上传、多模态材料处理、知识库构建、向量检索、推理接口接入和部署适配。团队在开发过程中逐步打通了从本地调试到完整部署的链路，使作品不只停留在功能原型层面，而是具备了可展示、可运行、可扩展的生产环境工程形态。

6.2 升级演进

从后续演进看，本作品仍有较清晰的升级路径。第一，可以继续扩充知识库来源和专题数据范围，增强系统对更多业务场景和更新材料的覆盖能力。第二，可以完善自由格式图谱解析的展示层，将当前解析结果进一步可视化，支持节点折叠、关系筛选、重点高亮和路径

回放，提高图谱可读性与交互性。第三，可以继续增强推荐与个性化能力，根据用户角色、历史操作和关注主题提供更有针对性的内容组织与任务提示。

在技术架构上，后续还可逐步推进移动端适配、缓存优化、异步任务处理、监控统计完善和模型调用策略优化，提高系统在高并发与长流程场景下的稳定性。在应用层面，也可继续完善发布、反馈、收藏、待办和搜索之间的数据联动，进一步强化平台化能力，使系统从“可用作品”演进为“可持续运营的产品”。

6.3 商业推广

从推广角度看，本作品具备较强的场景适配性和服务延展性。其核心能力并不局限于单一内容类型，而是适用于需要对复杂文本、图片材料和流程信息进行理解、检索、追问和承接的数字化场景。因此，在后续推广中，可以围绕信息服务平台、知识问答平台、内容运营平台和业务辅助平台等方向开展落地，形成“解析能力 + 智能交互 + 结果沉淀”的通用解决方案。

在商业模式上，作品具备按模块拆分和按场景交付的可能性。例如，可将自由格式图谱解析、智能问答、知识库检索、结果收藏待办、内容发布反馈等能力作为标准化功能组件进行组合；也可以面向不同使用单位提供定制化部署、知识库建设、界面适配和持续运维服务。随着知识资产积累和场景适配能力增强，作品具备从参赛成果进一步转化为可推广数字产品的潜力。

参考文献

[1] FastAPI Official Documentation. https://fastapi.tiangolo.com/
[2] LangGraph Official Documentation. https://langchain-ai.github.io/langgraph/
[3] Apache ECharts Official Documentation. https://echarts.apache.org/
[4] SQLModel Official Documentation. https://sqlmodel.tiangolo.com/
[5] ChromaDB Official Documentation. https://docs.trychroma.com/
[6] sentence-transformers Official Documentation. https://www.sbert.net/
[7] pdfplumber Official Documentation. https://github.com/jsvine/pdfplumber

[8] python-docx Official Documentation. https://python-docx.readthedocs.io/
[9] Docker Official Documentation. https://docs.docker.com/
[10] Nginx Official Documentation. https://nginx.org/en/docs/